Rapport – Projet OCaml

moule.informatique.univ-paris-diderot.fr/3lmtb/metrocaml

LORIOT Baptiste (baptiste.loriot@etu.u-paris.fr)
MONA Matthys (matthys.mona@etu.u-paris.fr)
BRUNO Thomas (thomas.bruno@etu.u-paris.fr)
DENQUIN Léo (leo.denquin-mathelot@etu.u-paris.fr)
MOSCARDI Lorenzo (lorenzo.moscardi@etu.u-paris.fr)
MEYNAUD Louis (louis.meynaud@etu.u-paris.fr)

1 Architecture Générale

- Organisation du projet : structure modulaire
- Trois niveaux d'abstraction :
  - **Librairie** `routing/` : noyau générique réutilisable
  - **Application** `metrocaml/` : implémentation métier
  - **Projet satellite** `tiktocaml/` : preuve de généricité
- Dépendances externes et infrastructure (Dune, OPAM)

2 Librairie de Routage : Fondations Génériques

2.1 Abstractions et Interfaces

La bibliothèque `routing/` est conçue pour être **totalement générique**, et utilisable dans n'importe quel contexte de graphe. Elle utilise des abstractions de haut niveau pour représenter les éléments fondamentaux d'un graphe, indépendamment de leur signification. Ainsi, elle manipule une architecture modulaire basée sur des interfaces telles que `GRAPH`, `NODE` et `EDGE`, plutôt que des types précis.

Ces modules sont définis par le projet utilisant la bibliothèque. Par exemple, *MétroCaml* définit les NODEs comme des stations, les EDGEs comme des liaisons de transport... tandis que *TiktoCaml* les définit respectivement comme des utilisateurs et des relations. La bibliothèque ne connaît pas ces détails, elle se contente de manipuler des éléments génériques.

2.2 Algorithmes

Routing fournit plusieurs algorithmes de parcours du graphe :

- **BFS** (parcours basique en largeur)
- **Dijkstra** (coût minimal) [1]
- **A*** (Dijkstra avec biais heuristique) [2]

Les algorithmes utilisent le principe du **state-space search** [3] : ils explorent un espace d'états défini par les nœuds du graphe, en utilisant une file de priorité pour gérer les nœuds à explorer.

2.3 Système de Pondération

Les algorithmes Dijkstra et A* nécessitent un système de pondération pour évaluer les coûts des chemins. La bibliothèque définit une interface `WEIGHT` qui permet de représenter une **métrique de coût**. Les projets utilisant la bibliothèque peuvent implémenter cette interface pour définir leurs propres critères de recherche. Ces modules peuvent alors utiliser les types spécifiques du projet. Par exemple, MétroCaml implémente des poids basés sur le temps de trajet, la distance, le nombre d'arrêts, ou les correspondances. TiktoCaml pourrait implémenter des poids basés sur la force de la relation, la fréquence d'interaction, etc.

Routing fournit aussi un système de **combinaison de poids**, `Combinator`, permettant de créer des métriques composites à partir de plusieurs critères. Par exemple, un utilisateur pourrait vouloir minimiser à la fois le temps de trajet et le nombre de correspondances, en pondérant ces deux critères selon ses préférences.

2.4 Résultats et Itinéraires

Les algorithmes de recherche retournent des résultats sous forme de `PATH`, une interface définie par la bibliothèque et paramétrée par les types `NODE` et `EDGE` du projet, qui contient la liste des arêtes parcourues.

2.5 Benchmarking et Performance

La bibliothèque inclut des outils de **benchmarking** pour évaluer la performance des différents algorithmes, soit sur un itinéraire spécifique, soit sur un nombre de cas de tests aléatoires. Ces outils permettent de mesurer le temps de calcul et la qualité des itinéraires proposés (en fonction des critères de pondération).

2.6 Démonstration de la généricité

La bibliothèque `routing` est ici utilisée dans deux contextes totalement différents : les **transports** (*MétroCaml*, le projet principal) et les **réseaux sociaux** (*TiktoCaml*, un mini-projet annexe). Cela prouve que la conception générique de la bibliothèque permet de l'adapter facilement à différents domaines, sans modifications structurelles majeures.

A | MetroCaml : Transports

- Nœuds = stations/arrêts
- Arêtes = trajets (bus, métro, marche, transfert...)
- Poids = critères (temps, distance, correspondances, nombre d'arrêts)

B | TiktoCaml : Réseaux Sociaux

- Nœuds = personnes
- Arêtes = relations d'amitié
- Poids = critères de parcours

3 Application MétroCaml

MétroCaml est un moteur de recherche d'itinéraires pour les transports. Il propose une interface complète pour trouver des trajets entre deux points d'un réseau en fonction de différents critères de recherche.

3.1 Modèle de Données

Pour charger des données réelles, MétroCaml s'appuie sur le standard **GTFS (General Transit Feed Specification)** [4]. Il s'agit d'un format universel de représentation des réseaux de transport, structuré sous la forme d'archives contenant des fichiers textes CSV. Nous avons choisi ce format car c'est le standard officiel en Europe et à l'international. Cette standardisation nous offre une grande flexibilité et permet à notre moteur de calculer des itinéraires sur des réseaux très variés, allant d'Île-de-France Mobilités à Corsica Ferries en passant par le réseau ferroviaire SNCF. Concrètement, une fois les données téléchargées, notre parseur lit ces fichiers **CSV** et les modélise directement en mémoire sous la forme d'un objet **TransportGraph**.

Pour offrir des itinéraires fiables et ancrés dans la réalité, le graphe statique est complété par un moteur d'ingestion de données en temps réel. **GTFS-RT** (*General Transit Feed Specification – Realtime*) [5] est l'extension temps réel du standard GTFS ; il encode les alertes et perturbations sous forme de flux Protocol Buffers. Ce module s'interface avec l'API PRIM d'Île-de-France Mobilités [6] pour récupérer ces flux et les intégrer via un **thread parallèle** afin de ne pas bloquer les modules de calcul d'itinéraires.

3.2 Interface Utilisateur

Deux interfaces sont disponibles au lancement de MétroCaml :

- une **interface console** (CLI) pour un affichage textuel
- une **interface graphique** (GUI) pour un affichage visuel interactif et l'intégration des alertes et perturbations en temps réel.

Les données peuvent être chargées en local ou en ligne.

3.3 Critères de Recherche et Profils

L'utilisateur a la possibilité de personnaliser sa recherche avec plusieurs critères sous forme de poids. Voici une liste exhaustive des différents critères :

- **Temps** : minimise l'heure d'arrivée
- **Distance** : privilégie la trajectoire géographique
- **Arrêts** : minimise le nombre de stations parcourues
- **Correspondances** : réduit le nombre de changements de lignes

L'utilisateur peut également utiliser des profils combinés prédéfinis comme **Rapide** et **Confort**, ou bien définir manuellement la pondération de chaque critère pour une recherche personnalisée.

3.4 Modes de Fonctionnement

- **Mode Local** : données pré-chargées, pas de connexion requise
- **Mode En Ligne** : données temps réel, APIs externes
- Réseaux disponibles : Île-de-France, Toulouse, Montpellier, Marseille, SNCF, Corsica
- Perturbations temps réel (notamment IDF) et accessibilité PMR

4 Processus de Développement

Répartition du travail au sein de l'équipe

- **Léo** : 18%
- **Baptiste** : 17%
- **Louis** : 17%
- **Thomas** : 17%
- **Matthys** : 16%
- **Lorenzo** : 15%

4.1 Gestion de Projet et Git

Le projet a suivi le workflow Git Flow :

- Branches `feature/[feature]` ou `[package]/[feature]` pour chaque développement, `fix/[fix]` pour les corrections de bug.
- Merge Requests (MR) dans `dev` obligatoires avec revue de code par au moins trois autres membres.
- Branche `dev` pour les sprints.
- Branche `main` uniquement pour les releases stables (MR unique `dev` → `main`).

Méthodologie agile avec des sprints et des releases, et un suivi des tâches via les tickets GitLab.

4.2 Intégration Continue (CI/CD)

Une pipeline GitLab CI a été mise en place pour le projet (`.gitlab-ci.yml`) :

- `setup` : Installation des dépendances via OPAM.
- `lint` : Analyse statique du code avec OCamlFormat pour garantir un code conforme.
- `test` : Exécution des tests avec Alcotest. Utilisation de Bisect_ppx pour mesurer la couverture de code, avec un seuil minimal imposé (à 70%). La pipeline délivre un artéfact contenant un fichier HTML avec le rapport de couverture, ainsi que le rapport XML pour l'intégration dans GitLab.
- `release` : Création automatique d'une release GitLab quand un tag est pushé.
- `docs` : Génération de la documentation avec OCamlDoc.

Nous voulions initialement publier automatiquement le rapport de couverture et la documentation sur GitLab Pages, mais cette fonctionnalité est désactivée sur le serveur Moule.

4.3 Tests et Qualité

- Suite de **tests unitaires** (avec *alcotest*)
- Benchmarking : comparaison des performances d'algorithmes / **tests de performance**
- Une fois un bug corrigé, un **test de régression** est ajouté.
- Couverture de code (avec *bisect_ppx*) : seuil minimal de 70% pour valider les MR.
- **Tests de compatibilité** :
  - Exécution du projet sur différents systèmes (Linux, Windows, MacOS)
  - Exécution de MétroCaml sur différents réseaux (IDF, Toulouse, etc.) pour vérifier la robustesse du parseur GTFS.

5 Statistiques – Benchmarks

Île-de-France

Benchmark unique

MétroCaml — de Moulin de Grais à Pépinière

Algorithme / Preset	Temps	Résultat
Dijkstra — Preset *Rapide*	26 786 ms	OK — 41 étapes · 2h15 · Arrivée 00:13 · 3 correspondances
A* — Preset *Rapide*	22 444 ms	OK — 41 étapes · 2h15 · Arrivée 00:13 · 3 correspondances
Dijkstra — Preset *Confort*	31 223 ms	OK — 41 étapes · 2h15 · Arrivée 00:13 · 3 correspondances
A* — Preset *Confort*	25 363 ms	OK — 41 étapes · 2h15 · Arrivée 00:13 · 3 correspondances
Dijkstra — Temps	16 653 ms	OK — 41 étapes · 2h15 · Arrivée 00:13 · 3 correspondances
A* — Temps	16 278 ms	OK — 41 étapes · 2h15 · Arrivée 00:13 · 3 correspondances
Dijkstra — Correspondances	38 377 ms	OK — 345 étapes · 35h44 · Arrivée 09:42 · 2 correspondances
A* — Correspondances	15 763 ms	OK — 79 étapes · 2h41 · Arrivée 00:39 · 4 correspondances
Dijkstra — Distance	62 793 ms	OK — 64 étapes · 45h13 · Arrivée 19:11 · 32 correspondances
A* — Distance	53 289 ms	OK — 71 étapes · 3h13 · Arrivée 01:11 · 9 correspondances
Dijkstra — Arrêts	3 453 ms	OK — 12 étapes · 31h21 · Arrivée 05:19 · 7 correspondances
A* — Arrêts	23 883 ms	OK — 37 étapes · 2h49 · Arrivée 00:47 · 9 correspondances
Arrêts (BFS)	2 936 ms	OK — 12 étapes · 34h17 · Arrivée 08:15 · 7 correspondances

Benchmark aléatoire

- **Catégorie :** Distance à vol d'oiseau entre le départ et l'arrivée
- **Dijkstra TW :** % des benchmarks où Dijkstra est plus rapide qu'A* en temps d'exécution
- **Dijkstra WW :** % des benchmarks où Dijkstra trouve un meilleur poids qu'A*
- **Dijkstra/A* (ms) :** Moyenne du temps d'exécution
- **Dijkstra/A* w :** Moyenne du poids trouvé

Catégorie	Bench	Dijk. TW	Dijk. WW	Dijk. (s)	A* (s)	Dijk. w	A* w
< 8000	2	50,0 %	75,0 %	0,6511	0,6809	0,5000	1,2500
[8000, 15000]	5	20,0 %	70,0 %	3,3885	2,3598	1,7000	1,9000
[15000, 25000]	10	30,0 %	85,0 %	8,0189	8,2559	2,3000	2,4000
[25000, 35000]	10	40,0 %	65,0 %	19,0318	16,8218	2,4500	2,9000
≥ 35000	23	26,1 %	76,1 %	41,2837	37,1742	2,9783	3,1522

Résumé global

- **Total des benchmarks :** 50
- **A* gagnant sur le temps d'exécution :** 35 cas
- **Dijkstra gagnant sur le poids :** 37 cas

A* est meilleur dans **70,00 %** des cas que Dijkstra en temps d'exécution, et dans **75,00 %** des cas Dijkstra a de meilleurs résultats en termes de poids (*Time / Transfers*).

Toulouse

Benchmark aléatoire

Catégorie	Bench	Dijk. TW	Dijk. WW	Dijk. (s)	A* (s)	Dijk. w	A* w
< 3000	111	38,7 %	88,3 %	0,0406	0,0343	0,1892	0,3333
[3000, 8000]	463	37,1 %	85,5 %	0,1231	0,1082	1,0961	1,3164
[8000, 13000]	581	36,5 %	86,4 %	0,2338	0,2054	1,7797	1,9707
[13000, 20000]	592	41,4 %	88,7 %	0,2919	0,2696	2,3454	2,4865
≥ 20000	253	40,7 %	91,3 %	0,3539	0,3237	3,1364	3,2411

Résumé global

- **Total des benchmarks :** 2000
- **A* gagnant sur le temps d'exécution :** 1225 cas
- **Dijkstra gagnant sur le poids :** 3505 cas (plusieurs presets testés par benchmark)

A* est meilleur dans **62,25 %** des cas que Dijkstra en temps d'exécution, et dans **87,62 %** des cas Dijkstra a de meilleurs résultats en termes de poids (*Time / Transfers*).

Conclusion Générale

Dijkstra obtient souvent de meilleurs résultats qu'A* en qualité d'itinéraire : il trouve un meilleur poids dans 75,00 % des cas sur IDF et 87,62 % des cas sur Toulouse. En revanche, A* est systématiquement plus rapide en temps de calcul (70,00 % et 62,25 % des cas), avantage particulièrement marqué sur les longues distances.

En résumé : **A* calcule plus vite, Dijkstra trouve plus souvent le meilleur itinéraire** selon les critères sélectionnés.

N.B. : Ces mesures ont été réalisées sur les graphes complets, sans exclusion de modes de transport, ce qui explique les temps de calcul relativement élevés, notamment sur IDFM (exploration de toutes les lignes de bus). En limitant le graphe, par exemple aux Métros/RER, on a des temps de calcul presque toujours inférieurs à 1 seconde.

Références

[1] Wang, Liang. *Dijkstra Algorithm*, Algorithm Blog, 22 nov. 2023, https://blog.ocaml.xyz/algorithm/2023/11/23/dijkstra-algorithm.html.

[2] *L'algorithme A* en OCaml*, http://decapode314.free.fr/dev/gen/a-star.html.

[3] Abbeel, Pieter. *Queue-Based Search*, UC Berkeley CS287, 16 nov. 2011, https://people.eecs.berkeley.edu/~pabbeel/cs287-fa11/slides/search.pdf.

[4] MobilityData, *GTFS Schedule Reference – General Transit Feed Specification*, https://gtfs.org/documentation/schedule/reference/, consulté en 2025.

[5] MobilityData, *GTFS Realtime Reference*, https://gtfs.org/documentation/realtime/reference/, consulté en 2025.

[6] Île-de-France Mobilités, *API PRIM – Plateforme Régionale d'Information Multimodale*, https://prim.iledefrance-mobilites.fr/, consulté en 2025.